

Fiche 303 — Objets, modes et attributs : ce dont R est fait

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — chapitre 3 « Objects, their modes and attributes » (§3.1 attributs intrinsèques, §3.2 changer la longueur, §3.3 lire et poser les attributs, §3.4 la classe)
Sources d'appoint	<i>R Language Definition</i> 4.6.1, chapitre 2 « Objects » — §2.1 types de base, §2.2 attributs, §2.3 objets composés spéciaux
Difficulté	Fondamental — la fiche qui explique <i>pourquoi</i> R se comporte comme il se comporte
Temps d'étude estimé	1 h 30
Prérequis	Fiches 300 à 302
Concepts clés	objet, structure atomique contre récursive , les six types de vecteurs, <code>mode()</code> contre <code>typeof()</code> contre <code>storage.mode()</code> , attributs intrinsèques , coercion et <code>as.quelquechose()</code> , allongement et troncature, <code>length(x) <- n</code> , <code>attributes()</code> / <code>attr()</code> , les attributs spéciaux <code>names</code> , <code>dim</code> , <code>dimnames</code> , <code>class</code> , <code>tsp</code> , ordre colonne-majeur , copie des attributs, <code>class()</code> et <code>unclass()</code> , <code>NULL</code> , environnements non copiés
À retenir en priorité	Une matrice est un vecteur plus un <code>dim</code> · <code>mode</code> ≠ <code>typeof</code> ≠ <code>storage.mode</code> · la coercion perd les attributs · les environnements ne sont pas copiés · <code>unclass()</code> pour voir ce qu'il y a dessous.

Vue d'ensemble

UN OBJET = DES DONNEES + DES ATTRIBUTS

ATOMIQUE (tous les elements du MEME mode)	RECURSIF (elements quelconques)
logical integer double	list
complex character raw	function expression

DEUX ATTRIBUTS INTRINSEQUES `mode(x)` `length(x)`

TROIS QUESTIONS, TROIS REPONSES

```
typeof(1:3)      "integer"    <- ce que R stocke VRAIMENT
mode(1:3)        "numeric"    <- au sens de Becker-Chambers-Wilks (S)
storage.mode(1:3) "integer"    <- pour appeler du C ou du Fortran
```

LES ATTRIBUTS SPECIAUX

```
names      etiqueter les elements
dim         FAIT d'un vecteur une matrice   (ordre COLONNE-MAJEUR)
dimnames    une LISTE de vecteurs de caracteres, elle-meme nommable
class       la base des methodes generiques ; modifiable SANS controle
tsp         start, end, frequency d'une serie temporelle
```

LA REGLE QUI RESUME TOUT

```
« Matrices and arrays are SIMPLY VECTORS with the attribute dim
  and optionally dimnames attached to the vector. »
```

COPIE DES ATTRIBUTS

```
fonction scalaire  -> preserve
operation binaire   -> copie depuis le PLUS LONG
sous-ensemble      -> LAISSE TOMBER tout sauf names/dim/dimnames
sous-assignation   -> preserve
coercition         -> LAISSE TOUT TOMBER
```

Le problème posé. « *The entities R operates on are technically known as **objects**.* » (§3.1) Jusqu'ici, les vecteurs semblaient être *les* données. Ce chapitre montre qu'un objet R est en réalité **deux choses** : des données brutes, et **un jeu d'attributs** qui dit comment les lire. C'est de là que vient l'unité du langage — et c'est aussi ce qui explique les comportements les plus déroutants des fiches précédentes.

INTUITION — LA PHRASE QUI VAUT TOUT LE CHAPITRE.

« ***Matrices and arrays are simply vectors with the attribute `dim` and optionally `dimnames` attached to the vector.*** » (*R Language Definition* §2.2) Il n'y a **pas** de type matrice en R. Il y a des vecteurs, et un attribut qui dit « lis-moi en 10 par 10 ». C'est pourquoi `m[1]` a un sens (fiche 302), pourquoi `length(m)` rend le nombre de **cases**, et pourquoi `attr(z, "dim") <- c(10,10)` suffit à transformer un vecteur en matrice.

● Concept 1 — Atomique contre récursif

DÉFINITION (§3.1).

*Les objets sont par exemple des vecteurs de valeurs **numériques (réelles)** ou **complexes**, des vecteurs de valeurs **logiques**, des vecteurs de **chaînes de caractères**. *« These are known as **"atomic" structures** since **their components are all of the same type, or mode**, namely `numeric` , `complex` , `logical` , `character` and `raw` . »**

Règle (§3.1). « *Vectors must have their values all of the same mode. Thus any given vector must be unambiguously either logical, numeric, complex, character or raw.* »

*« (The only **apparent** exception to this rule is the special "value" listed as `NA` for quantities not available, but **in fact there are several types of NA** .) »* — le complément déjà rencontré en fiche 301 : `NA` n'est pas une entorse à la règle, c'est **une famille de valeurs typées**.

Un vecteur vide a quand même un mode (§3.1). *« Note that a vector **can be empty and still have a mode**. For example the empty character string vector is listed as `character(0)` and the empty numeric vector as `numeric(0)` . »* Le vide n'est pas l'absence de type — d'où l'affichage `character(0)` plutôt qu'un silence.

1.1 Les structures récursives

DÉFINITION (§3.1).

*« R also operates on objects called **lists**, which are of mode `list` . These are **ordered sequences of objects** which **individually can be of any mode**. Lists are known as **"recursive" rather than atomic** structures since **their components can themselves be lists in their own right**. »*

« *The other recursive structures are those of mode `function` and `expression` .* »

Famille	Modes	Propriété
Atomique	logical , numeric , complex , character , raw	tous les éléments du même mode
Récuratif	list , function , expression	les éléments peuvent être n'importe quoi , y compris des objets de la même sorte

« **Functions** are the objects that form part of the R system along with similar user written functions » — la fiche 309. « **Expressions** as objects form an advanced part of R which will not be discussed in this guide, except indirectly when we discuss formulae used with modeling » — la fiche 317.

COMPLÉMENT (R LANGUAGE DEFINITION §2.1.2).

« **Lists are vectors**, and the basic vector types are referred to as **atomic vectors** where it is necessary to exclude lists. » Une liste **est** un vecteur : un « vecteur générique ». C'est pourquoi `length()` , `[` , `names()` marchent dessus exactement comme sur un vecteur numérique.

1.2 Les deux attributs intrinsèques

DÉFINITION (§3.1).

« By the **mode** of an object we mean **the basic type of its fundamental constituents**. This is a special case of a "property" of an object. **Another property of every object is its `Length`**. The functions `mode(object)` and `Length(object)` can be used to find out the mode and length of any defined structure. »

*« Further properties of an object are usually provided by `attributes(object)` . **Because of this, `mode` and `length` are also called "intrinsic attributes" of an object.** »*

L'exemple du cours. « if `z` is a complex vector of length 100, then in an expression `mode(z)` is the character string `"complex"` and `Length(z)` is 100. »

La note 2 du §3.1 tempère. « Note however that `Length(object)` **does not always contain intrinsic useful information**, e.g. when `object` is a function. »

La note 1 du §3.1 annonce le concept 2. « *numeric mode is actually an amalgam of two distinct modes, namely integer and double precision* ». C'est la raison d'être de `typeof()` et de `storage.mode()`.

● Concept 2 — `mode`, `typeof`, `storage.mode` : trois questions différentes

Le cours d'introduction ne parle que de `mode`. La *R Language Definition* (chapitre 2) explique pourquoi cela ne suffit pas.

DÉFINITION (R LANGUAGE DEFINITION §2).

*« The R specific function `typeof` returns the type of an R object. Note that in the C code underlying R, **all objects are pointers to a structure with `typedef SEXP`**; the different R data types are represented in C by `SEXP`, which determines how the information in the various parts of the structure is used. »*

*« Function `mode` gives information about the mode of an object **in the sense of Becker, Chambers & Wilks (1988)**, and is **more compatible with other implementations of the S language**. Finally, the function `storage.mode` returns the storage mode ... **It is generally used when calling functions written in another language, such as C or FORTRAN**, to ensure that R objects have the data type expected by the routine being called. (In the S language, vectors with integer or real values are both of mode `"numeric"`, so their storage modes need to be distinguished.) »*

L'exemple du manuel, qui montre les trois réponses sur le même objet :

```
x <- 1:3
typeof(x)      # "integer"
mode(x)        # "numeric"
storage.mode(x) # "integer"
```

Fonction	Répond à la question	Sert à
<code>typeof()</code>	<i>qu'est-ce que R stocke vraiment ?</i>	comprendre, déboguer
<code>mode()</code>	<i>quel est le type au sens de S ?</i>	la compatibilité, le code hérité
<code>storage.mode()</code>	<i>sous quelle forme mémoire ?</i>	appeler du C ou du Fortran (fiche 320)

2.1 Les six types de vecteurs

Règle (§2.1.1). *« R has **six basic ("atomic") vector types** : `logical` , `integer` , `real` , `complex` , `character` (in C aka `string`) and `raw` . »*

<code>typeof</code>	<code>mode</code>	<code>storage.mode</code>
<code>logical</code>	<code>logical</code>	<code>logical</code>
<code>integer</code>	<code>numeric</code>	<code>integer</code>
<code>double</code>	<code>numeric</code>	<code>double</code>
<code>complex</code>	<code>complex</code>	<code>complex</code>
<code>character</code>	<code>character</code>	<code>character</code>
<code>raw</code>	<code>raw</code>	<code>raw</code>

Les deux lignes qui expliquent tout sont `integer` et `double` : deux `typeof` distincts pour **un seul** `mode` . C'est exactement la note 1 du §3.1 d'*An Introduction to R* — « `numeric` mode is actually an amalgam of two distinct modes ».

La phrase qui ferme la question (§2.1.1). « *Single numbers, such as `4.2` , and strings, such as "four point two" are still vectors, of length 1 ; there are no more basic types. Vectors with length zero are possible (and useful).* »

« Vectors can be thought of as **contiguous cells containing data**. Cells are accessed through **indexing operations** such as `x[5]` . »

2.2 Le tableau complet des `typeof`

NOTATION (R LANGUAGE DEFINITION §2).

« The following table describes the possible values returned by `typeof` and what they are. »

typeof	Ce que c'est
"NULL"	NULL
"symbol"	un nom de variable
"pairlist"	un objet pairlist (<i>mainly internal</i>)
"closure"	une fonction
"environment"	un environnement
"promise"	un objet servant à l'évaluation paresseuse
"language"	une construction du langage R
"special"	une fonction interne qui n'évalue pas ses arguments
"builtin"	une fonction interne qui évalue ses arguments
"char"	une chaîne « scalaire » (interne seulement)
"logical"	un vecteur de valeurs logiques
"integer"	un vecteur de valeurs entières
"double"	un vecteur de valeurs réelles
"complex"	un vecteur de valeurs complexes
"character"	un vecteur de valeurs caractères
"..."	la variable spéciale à nombre variable d'arguments
"any"	un type spécial qui apparie tous les types — « <i>there are no objects of this type</i> »
"expression"	un objet expression
"list"	une liste
"bytecode"	du code-octet (<i>interne seulement</i>)
"externalptr"	un pointeur externe
"weakref"	une référence faible

typeof	Ce que c'est
"raw"	un vecteur d'octets
"S4"	un objet S4 qui n'est pas un objet simple

« Users **cannot easily get hold of objects** of types marked with a `***`. »

REMARQUE DE LECTURE.

Cette table est la carte du langage entier. Trois entrées méritent d'être repérées dès maintenant, parce que les fiches suivantes s'appuient dessus : `closure` (une fonction est un objet — fiche 309), `promise` (l'évaluation paresseuse — fiche 309), `environment` (la portée — fiche 310).

● Concept 3 — La coercion

Règle (§3.1). « *R caters for **changes of mode almost anywhere it could be considered sensible to do so** (and a few where it might not be).* »

L'aller-retour du cours :

```
z <- 0:9
digits <- as.character(z) # c("0", "1", "2", ..., "9")
d <- as.integer(digits)   # « Now d and z are the same. »
```

« There is a **large collection of functions of the form** `as.something()` for **either coercion from one mode to another, or for investing an object with some other attribute it may not already possess**. The reader should consult the different help files to become familiar with them. »

Le piège numérique, en note 3 du §3.1. « *In general, coercion from numeric to character and back again will not be exactly reversible, because of roundoff errors in the character representation.* »

L'aller-retour du cours est réversible **parce que 0:9 sont des entiers**. Avec des réels, l'écriture en caractères passe par un arrondi d'affichage, et le retour ne redonne pas exactement le même nombre. **Ne jamais faire transiter des réels par une chaîne** pour les manipuler.

La coercion efface tout (R Language Definition §2.2.6). « *Coercion drops all attributes.* » Coercer une matrice, c'est perdre son `dim` ; coercer un facteur, c'est perdre ses `levels` . C'est la règle la plus radicale du chapitre — et le concept 6 la remet dans son contexte.

ENRICHISSEMENT PÉDAGOGIQUE (HORS COURS) — LA HIÉRARCHIE DE COERCITION IMPLICITE.

An Introduction to R décrit la coercion **explicite** (`as. ....`). Il ne donne pas la règle de la coercion **implicite**, celle qui s'applique quand `c()` reçoit des modes mélangés. Elle se déduit néanmoins de la règle du §3.1 (« un vecteur doit être d'un seul mode ») : R choisit **le mode le plus général capable de tout représenter**, dans l'ordre

logical → integer → double → complex → character

Ainsi `c(TRUE, 1L)` est entier, `c(1L, 2.5)` est double, et `c(1, "a")` **est caractère** — le `1` devient `"1"` . C'est la cause la plus fréquente d'une colonne « qui devrait être numérique » et ne l'est pas. **Ce raisonnement n'est pas dans le cours ; il en est la conséquence.**

● Concept 4 — Changer la longueur d'un objet

Règle (§3.2). « *An "empty" object may still have a mode.* »

```
e <- numeric()      # un vecteur VIDE de mode numerique
```

4.1 Allonger — en écrivant hors des bornes

MÉTHODE (§3.2).

« Once an object of any size has been created, **new components may be added to it simply by giving it an index value outside its previous range.** »

```
e[3] <- 17
```

*« now makes `e` a vector of **length 3**, (the first two components of which are at this point both `NA`). »*

La condition (§3.2). « This applies to **any structure at all, provided the mode of the additional component(s) agrees with the mode of the object in the first place.** » Si le mode ne concorde pas, il n'y a pas d'erreur — il y a **coercition de tout l'objet** (concept 3).

« This **automatic adjustment of lengths** of an object is used often, for example **in the** `scan()` **function for input** » — fiche 307.

4.2 Raccourcir — par assignation ou par `length<-`

MÉTHODE (§3.2).

« Conversely, **to truncate the size of an object requires only an assignment** to do so. »

```
alpha <- alpha[2 * 1:5] # les composantes d'INDICE PAIR ; longueur 10 -> 5
length(alpha) <- 3      # on ne garde que les trois premieres
```

*« makes it an object of length 5 consisting of just the former components **with even index.** (The old indices are not retained, of course.) »* Puis : « and **vectors can be extended (by missing values) in the same way** » — `length(alpha) <- 20` allonge, en remplissant de `NA`.

CONTRÔLE — VÉRIFIER $2 * 1:5$.

Le `:` étant prioritaire (fiche 301), $2 * 1:5$ vaut $2 * c(1,2,3,4,5) = c(2,4,6,8,10)$ — bien **les indices pairs**. Écrit $2 * (1:5)$, ce serait identique ; écrit $(2*1):5$, ce serait $2:5$. Le cours choisit l'écriture courte, mais elle repose entièrement sur la règle de priorité.

`length(x) <- n` est une fonction de remplacement (fiche 302) : ``length<-`(x, value = n)`. Elle tronque si `n` est plus petit, allonge par des `NA` s'il est plus grand.

● Concept 5 — Les attributs

DÉFINITION (§3.3).

« The function `attributes(object)` returns a **list of all the non-intrinsic attributes** currently defined for that object. The function `attr(object, name)` can be used to select a specific attribute. »

« These functions are **rarely used**, except in rather special circumstances when some new attribute is being created for some particular purpose, for example **to associate a creation date or an operator with an R object**. The concept, however, is very important. »

« **Some care should be exercised when assigning or deleting attributes** since they are **an integral part of the object system used in R**. »

L'exemple qui fabrique une matrice à partir de rien (§3.3) :

```
attr(z, "dim") <- c(10, 10)
```

« allows R to **treat** `z` **as if it were a 10-by-10 matrix**. » Une ligne, un attribut — et `z` n'a pas bougé d'un octet.

COMPLÉMENT (R LANGUAGE DEFINITION §2.2).

*« **All objects except `NULL` can have one or more attributes** attached to them. Attributes are **stored as a pairlist where all elements are named**, but should be thought of as **a set of `name=value` pairs**. A listing of the attributes can be obtained using `attributes` and set by `attributes<-` ; individual components are accessed using `attr` and `attr<-` . »*

« Some attributes have **special accessor functions** (e.g. `levels<-` for factors) and **these should be used when available**. In addition to **hiding details of implementation** they may **perform additional operations**. **R attempts to intercept calls to `attr<-` and to `attributes<-` that involve the special attributes and enforces the consistency checks.** »

La règle pratique : passer par l'accesseur dédié (`names<-` , `dim<-` , `levels<-` , `class<-`) plutôt que par `attr<-` . Non par élégance, mais parce que **l'accesseur vérifie la cohérence** et que `attr<-` ne le fait qu'en partie.

5.1 Les cinq attributs spéciaux

names (§2.2.1). *« A `names` attribute, **when present, labels the individual elements of a vector or list**. When an object is printed the `names` attribute, when present, **is used to label the elements**. The `names` attribute **can also be used for indexing purposes**, for example `quantile(x)["25%"]` . »*

« One may get and set the names using `names` and `names<-` constructions. **The latter will perform the necessary consistency checks** to ensure that the `names` attribute has **the proper type and length.** »

« **Pairlists and one-dimensional arrays are treated specially**. For pairlist objects, **a virtual `names` attribute is used** ; the `names` attribute is really **constructed from the tags** of the list components. **For one-dimensional arrays the `names` attribute really accesses `dimnames[[1]]`** . »
— le complément exact de ce qu'annonçait la fiche 302 sur les tableaux à une dimension.

dim (§2.2.2). « The `dim` attribute is used to **implement arrays**. The content of the array is **stored in a vector in column-major order** and the `dim` attribute is **a vector of integers specifying the respective extents** of the array. **R ensures that the length of the vector is the product of the lengths of the dimensions**. The length of **one or more dimensions may be zero.** »

« **A vector is not the same as a one-dimensional array** since the latter **has a `dim` attribute of length one**, whereas the former **has no `dim` attribute**. »

EN CLAIR — « COLONNE-MAJEUR ».

Une matrice 2×2 remplie de `1:4` se lit **colonne par colonne** :

$$\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

C'est ce que montrait l'exemple `matrix(1:4, 2)` de la fiche 302. C'est aussi ce qui rend `m[1]` prévisible : le premier élément **du vecteur sous-jacent** est en haut à gauche, le deuxième **en dessous**, pas à droite.

`dimnames` (§2.2.3). « Arrays **may name each dimension separately** using the `dimnames` attribute which is **a list of character vectors**. The `dimnames` list may itself have names, which are then **used for extent headings when printing arrays**. » — l'explication de l'en-tête d'un `table()`.

`class` (§2.2.4). « R has an **elaborate class system**, principally controlled via the `class` attribute. This attribute is **a character vector containing the list of classes that an object inherits from**. This forms the basis of the "generic methods" functionality in R. »

L'avertissement le plus fort du chapitre. « This attribute **can be accessed and manipulated virtually without restriction by users**. There is no checking that an object **actually contains the components that class methods expect**. Thus, **altering the `class` attribute should be done with caution**, and **when they are available specific creation and coercion functions should be preferred**. »

Poser `class(x) <- "data.frame"` sur un objet quelconque ne le rend pas data frame : cela **fait chercher à R les méthodes des data frames** sur un objet qui n'a pas leurs composantes. La panne survient **au premier appel générique**, loin de la ligne fautive.

`tsp` (§2.2.5). « The `tsp` attribute is used to hold **parameters of time series : start, end, and frequency**. This construction is mainly used to handle **series with periodic substructure** such as monthly or quarterly data. »

Énoncé. Partant de `z <- 1:100`, en faire une matrice 10×10 en n'utilisant que des attributs. Puis revenir au vecteur.

Étape 1 — l'état initial. `z` est un vecteur `integer` de longueur 100, **sans aucun attribut** : `attributes(z)` rend `NULL`.

Étape 2 — poser `dim`, exactement comme au §3.3.

```
attr(z, "dim") <- c(10, 10)
```

« allows R to treat `z` as if it were a 10-by-10 matrix ».

Étape 3 — vérifier la contrainte de cohérence. « R ensures that **the length of the vector is the product of the lengths of the dimensions** » (§2.2.2). Ici $10 \times 10 = 100 = \text{length}(z)$: la pose réussit. Avec `c(10, 11)`, elle échouerait — c'est le contrôle de cohérence dont parle le §2.2.

Étape 4 — constater ce qui a changé. Rien, dans les données. `typeof(z)` vaut toujours `"integer"`, `length(z)` toujours `100`. Mais `class(z)` rend désormais `"matrix"` `"array"`, et l'affichage devient tabulaire. **L'attribut a changé la lecture, pas le contenu.**

Étape 5 — l'ordre de remplissage. « The content of the array is stored in a vector **in column-major order** ». Donc `z[1,1]` vaut 1, `z[2,1]` vaut 2, et `z[1,2]` vaut 11. C'est bien la **première colonne** qui se remplit d'abord.

Étape 6 — l'écriture recommandée. Le §2.2 le dit : « some attributes have **special accessor functions** ... **these should be used when available** ». La forme préférable est donc

```
dim(z) <- c(10, 10)
```

qui est ``dim<` (z, value = c(10,10))` (fiche 302) et applique les mêmes contrôles de façon explicite.

Étape 7 — revenir au vecteur. Il suffit de **retirer l'attribut**, en lui assignant `NULL` :

```
dim(z) <- NULL      # z redevient un vecteur de longueur 100
```

Étape 8 — la leçon. La matrice n'a jamais existé comme **type**. Elle a existé comme **lecture**. C'est cela, la phrase du §2.2 : « Matrices and arrays are **simply vectors** with the attribute `dim` ... attached ».

● Concept 6 — Quand les attributs survivent, et quand ils disparaissent

Règle (R Language Definition §2.2.6). « *Whether attributes should be copied when an object is altered is **a complex area**, but **there are some general rules*** (Becker, Chambers & Wilks, 1988, pp. 144–6). »

Opération	Ce qui arrive aux attributs
Fonction scalaire — « <i>those which operate element-by-element on a vector and whose output is similar to the input</i> »	*« should preserve attributes (except perhaps <code>class</code>) »*
Opération binaire	« <i>normally copy most attributes from the longer argument (and if they are of the same length from both, preferring the values on the first)</i> ». Ici « <i>most</i> » signifie tout sauf <code>names</code> , <code>dim</code> et <code>dimnames</code> , qui sont posés comme il convient par le code de l'opérateur
Sous-ensemble (autre que par un index vide)	*« generally drops all attributes except <code>names</code> , <code>dim</code> and <code>dimnames</code> , which are reset as appropriate »*
Sous-assignation	« <i>generally preserves attributes even if the length is changed</i> »
Coercition	« <i>drops all attributes</i> »
Tri (méthode par défaut)	*« drops all attributes except <code>names</code> , which are sorted along with the object »*

EN CLAIR — POURQUOI CETTE TABLE EST UTILE.

Elle répond à la question « où sont passés mes attributs ? » avant même qu'on se la pose. Les deux lignes à retenir sont **la coercition** (elle efface **tout**) et **le sous-ensemble** (il n'épargne que `names`, `dim`, `dimnames`). Un attribut maison — une date de création, une unité de mesure — **ne survit pas à un** `x[1:5]`.

Et la ligne **sous-assignation** est son contraire exact : `x[1:5] <- v` **préserve** les attributs, « *even if the length is changed* ». Lire et écrire ne se comportent pas pareil.

● Concept 7 — La classe

DÉFINITION (§3.4).

*« All objects in R have a **class**, reported by the function `class` . For simple vectors this is just the **mode** — for example `"numeric"` , `"logical"` , `"character"` or `"list"` — but `"matrix"` , `"array"` , `"factor"` and `"data.frame"` are other possible values. »*

DÉFINITION (§3.4).

« A special attribute known as the **class of the object** is used to allow for an **object-oriented style of programming in R**. For example if an object has class `"data.frame"` , it will be printed in a certain way, the `plot()` function will display it graphically **in a certain way**, and other so-called **generic functions** such as `summary()` will **react to it as an argument in a way sensitive to its class**. »

C'est la mécanique déjà observée à l'annexe A (fiche 300) : `plot(Expt, Speed)` produit des boîtes à moustaches **parce que** `Expt` **est un facteur**. La fonction est la même ; **la classe choisit la méthode**. Voir fiche 311.

« A different style using **"formal" or "S4" classes** is provided in package `methods` . » (note 4)

7.1 `unclass()` — regarder dessous

MÉTHODE (§3.4).

« To remove temporarily the effects of class, use the function `unclass()` . »

```
winter          # affiche « in data frame form, which is rather like a matrix »
unclass(winter)  # affiche « as an ordinary list »
```

« Only in rather special situations do you need to use this facility, but one is when you are learning to come to terms with the idea of class and generic functions. »

MÉTHODE — L'USAGE PÉDAGOGIQUE QUE LE COURS RECOMMANDE EXPLICITEMENT.

`unclass()` est l'**outil d'apprentissage** du chapitre : il montre qu'un data frame **est une liste**, qu'un facteur **est un vecteur d'entiers**, qu'une matrice **est un vecteur**. Chaque fois qu'un objet R paraît magique, `unclass()` — ou `attributes()` — dissipe la magie.

● Concept 8 — Trois objets à connaître dès maintenant

8.1 NULL — l'absence, pas le vide

DÉFINITION (R LANGUAGE DEFINITION §2.1.6).

*« There is a special object called `NULL`. It is used **whenever there is a need to indicate or specify that an object is absent**. It should not be confused with a vector or list of zero length. »*

*« The `NULL` object **has no type and no modifiable properties**. There is only one `NULL` object in R, to which all instances refer. To test for `NULL` use `is.null`. **You cannot set attributes on `NULL`**. »*

	NULL	numeric(0)
Ce que c'est	l'absence d'objet	un vecteur numérique vide
A-t-il un type ?	non	oui — <code>"double"</code>
<code>length()</code>	0	0
Peut porter des attributs ?	non	oui

Les deux ont une **longueur nulle**, et ce sont **deux choses différentes**. C'est pourquoi `x$inexistant` rend `NULL` (fiche 302) et non `character(0)` : l'élément **n'est pas là**, il n'est pas *vide*.

8.2 Les environnements — la seule chose que R ne copie pas

Règle (R Language Definition §2.1.10). « *Unlike most other R objects, environments are not copied when passed to functions or used in assignments. Thus, if you assign the same environment to several symbols and change one, the others will change too. In particular, assigning attributes to an environment can lead to surprises.* »

C'est l'**exception à la sémantique de copie** décrite en fiche 302 (le mécanisme ``*tmp*``). Tout le reste de R se comporte par valeur ; les environnements se comportent **par référence**. La conséquence est double : ils servent à contourner l'absence d'effet de bord quand on en a besoin, et ils produisent des bugs impossibles à comprendre quand on ne sait pas qu'ils sont là. Voir fiche 310.

*« Environments can be thought of as consisting of two things : **a frame**, consisting of a set of **symbol-value pairs**, and **an enclosure**, a pointer to an enclosing environment. ... **Environments form a tree structure** in which the enclosures play the role of parents. The tree of environments **is rooted in an empty environment**, available through `emptyenv()`, which has no parent. It is the direct parent of **the environment of the base package** (`baseenv()`). »*

8.3 Les promesses — pourquoi les arguments ne sont pas évalués tout de suite

DÉFINITION (§2.1.8).

« **Promise objects are part of R's lazy evaluation mechanism.** They contain **three slots : a value, an expression, and an environment.** When a function is called the arguments are matched and then **each of the formal arguments is bound to a promise.** The expression that was given for that formal argument **and a pointer to the environment the function was called from** are stored in the promise. »

« **Until that argument is accessed there is no value associated with the promise.** When the argument is accessed, **the stored expression is evaluated in the stored environment**, and the result is returned. **The result is also saved by the promise.** »

C'est le mécanisme qui explique qu'un argument jamais utilisé puisse être invalide sans conséquence, et que `substitute` puisse retrouver **le texte** de ce qui a été passé : « The `substitute` function will **extract the content of the expression slot** » (fiche 317).

« There is **generally no way in R code to check whether an object is a promise or not**, nor is there a way to use R code to determine the environment of a promise. »

● Concept 9 — Les objets composés spéciaux

Le chapitre 2 de la *R Language Definition* se clôt sur deux structures que les fiches 304 et 306 traiteront en détail. Leur **définition en termes d'attributs** appartient toutefois à cette fiche-ci.

DÉFINITION — UN FACTEUR (§2.3.1).

« Factors are used to describe items that can have **a finite number of values** (gender, social class, etc.). **A factor has a `levels` attribute and class `"factor"`** . Optionally, it may also contain a **`contrasts`** attribute which controls the parametrisation used when the factor is used in a modeling function. »

*« A factor may be **purely nominal** or may have **ordered categories**. In the latter case, it should be defined as such and have a **class vector** `c("ordered", "factor")` . »*

« Factors are currently implemented using **an integer array to specify the actual levels** and a second array of names that are mapped to the integers. **Rather unfortunately users often make use of the implementation in order to make some calculations easier. This, however, is an implementation issue and is not guaranteed to hold in all implementations of R.** »

Cette mise en garde est la source du piège de la fiche 302 — indexer par un facteur revient à `x[as.integer(i)]` . Le manuel dit ici que **s'appuyer là-dessus est une faute**, même si cela marche.

DÉFINITION — UN DATA FRAME (§2.3.2).

« Data frames are the R structures which **most closely mimic the SAS or SPSS data set**, i.e. a **"cases by variables" matrix of data**. »

« **A data frame is a list of vectors, factors, and/or matrices all having the same length** (number of rows in the case of matrices). In addition, a data frame generally has a `names` attribute labeling **the variables** and a `row.names` attribute for labeling **the cases**. »

« A data frame **can contain a list** that is the same length as the other components. The list can contain elements of **differing lengths**, thereby providing a data structure for **ragged arrays**. **However, as of this writing such arrays are not generally handled correctly.** »

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« quel est le type de cet objet ? »	trois réponses possibles : <code>typeof</code> , <code>mode</code> , <code>storage.mode</code>
« pourquoi <code>mode</code> dit <code>numeric</code> et <code>typeof</code> dit <code>integer</code> ? »	<code>numeric</code> est un amalgame d' <code>integer</code> et de <code>double</code>
« je prépare un appel à du C »	<code>storage.mode()</code>
« ma colonne numérique est devenue du texte »	coercition implicite : un vecteur n'a qu'un seul mode
« mon aller-retour numérique ne redonne pas la même valeur »	« <i>will not be exactly reversible, because of roundoff errors</i> »
« où sont passés mes attributs ? »	table de la copie des attributs — coercion et sous-ensemble effacent
« comment faire une matrice à partir d'un vecteur ? »	poser <code>dim</code> — « <i>matrices are simply vectors with the attribute <code>dim</code></i> »
« pourquoi <code>m[1,2]</code> n'est pas le deuxième élément ? »	ordre colonne-majeur
« <code>length()</code> de ma matrice rend le mauvais nombre »	il rend le nombre de cases — le produit des extensions
« pourquoi mon objet s'affiche bizarrement ? »	son <code>class</code> — voir dessous avec <code>unclass()</code>
« je change la classe et tout casse »	« <i>there is no checking that an object actually contains the components that class methods expect</i> »
« allonger un vecteur »	écrire hors des bornes , ou <code>length(x) <- n</code>
« les premiers éléments sont <code>NA</code> »	l'allongement remplit de <code>NA</code>
« <code>NULL</code> ou vecteur vide ? »	absence contre vide typé — <code>is.null()</code>
« je modifie un objet dans une fonction et l'original change »	c'est un environnement — la seule chose non copiée
« à quoi sert <code>substitute</code> ? »	à lire la fente expression d'une promesse

Comment résoudre ce type d'exercice

Protocole « disséquer un objet inconnu » — 5 étapes.

1. `class(x)` — comment R va le traiter.
2. `typeof(x)` — ce qu'il est réellement en mémoire.
3. `length(x)` — combien d'éléments **du vecteur sous-jacent**.
4. `attributes(x)` — tout ce qui n'est pas intrinsèque : `dim` , `names` , `levels` , `class` ...
5. `unclass(x)` ou `str(x)` — regarder dessous. « *one [situation] is when you are learning to come to terms with the idea of class and generic functions* ».

Protocole « poser ou retirer un attribut » — 4 étapes.

1. Préférer l'**accesseur dédié** (`names<-` , `dim<-` , `levels<-` , `class<-`) à `attr<-` : il **vérifie la cohérence** et peut faire davantage.
2. Vérifier la **contrainte de longueur** : pour `dim` , « *the length of the vector is **the product** of the lengths of the dimensions* ».
3. Pour **retirer** un attribut, lui assigner `NULL` .
4. Se rappeler que `NULL` **n'accepte aucun attribut** et qu'un attribut maison **ne survit pas à un sous-ensemble**.

Protocole « diagnostiquer une coercion non voulue » — 4 étapes.

1. `typeof()` avant et après l'opération suspecte.
2. Chercher le **mélange de modes** : un seul `"a"` dans un `c()` fait passer **tout** le vecteur en caractère.
3. Vérifier si l'opération est **de la famille qui efface** : coercion (tout), sous-ensemble (tout sauf `names` / `dim` / `dimnames`), tri (tout sauf `names`).
4. Corriger **à la source** — au moment de la construction du vecteur — plutôt qu'après coup par un `as.numeric()` qui peut introduire des `NA` .

● Common mistakes

Erreur	Correction
Croire que <code>mode</code> et <code>typeof</code> disent la même chose	<code>mode</code> amalgame <code>integer</code> et <code>double</code> sous <code>numeric</code>
Utiliser <code>mode()</code> pour préparer un appel C	c'est <code>storage.mode()</code>
Croire qu'un vecteur peut mélanger les modes	« <i>must have their values all of the same mode</i> » — sinon coercition
Croire qu'un vecteur vide n'a pas de mode	<code>character(0)</code> , <code>numeric(0)</code> — le vide est typé
Faire transiter des réels par une chaîne	« <i>not exactly reversible, because of roundoff errors</i> »
S'attendre à retrouver ses attributs après un <code>as...</code>	« <i>coercion drops all attributes</i> »
S'attendre à les retrouver après <code>x[1:5]</code>	seuls <code>names</code> , <code>dim</code> , <code>dimnames</code> survivent
Croire que la sous-assignation efface comme le sous-ensemble	elle préserve , « <i>even if the length is changed</i> »
Croire qu'une matrice est un type	c'est un vecteur + <code>dim</code>
Attendre un remplissage ligne par ligne	l'ordre est colonne-majeur
Croire <code>length(m)</code> égal au nombre de lignes	c'est le produit des extensions
Confondre vecteur et tableau à une dimension	le second a un <code>dim</code> de longueur 1
Poser <code>dim</code> incohérent avec la longueur	« <i>R ensures that the length ... is the product</i> » → erreur
Utiliser <code>attr<-</code> là où existe un accesseur	l'accesseur contrôle la cohérence et fait parfois plus
Changer <code>class</code> pour « convertir » un objet	« <i>there is no checking ...</i> » — panne différée
Croire <code>NULL</code> équivalent à un vecteur vide	« <i>should not be confused</i> with a vector or list of zero length »
Poser un attribut sur <code>NULL</code>	impossible
Croire que tout est copié en R	les environnements ne le sont pas
Compter sur le codage entier des facteurs	« <i>an implementation issue ... not guaranteed</i> »

Erreur	Correction
Croire qu'un data frame est une matrice	c'est une liste de vecteurs de même longueur
Croire <code>length()</code> toujours informatif	« <i>does not always contain intrinsic useful information, e.g. when object is a function</i> »

Ultimate Review

Atomique contre récursif. Atomique : `logical`, `numeric`, `complex`, `character`, `raw` — « *their components are **all of the same type, or mode*** ». Récursif : `list`, `function`, `expression` — « *their components can themselves be lists in their own right* ». **Une liste est un vecteur** (« vecteur générique ») ; on dit *vecteur atomique* quand il faut exclure les listes. **Un vecteur vide a un mode** : `character(0)`, `numeric(0)`.

Attributs intrinsèques. `mode()` et `length()`. « *Because of this, mode and length are also called **intrinsic attributes*** ». `length()` n'est pas toujours informatif (une fonction).

Trois fonctions, trois questions. `typeof` = ce que R stocke (`SEXPTYPE`) · `mode` = au sens de Becker-Chambers-Wilks, compatible S · `storage.mode` = pour **C et Fortran**. Sur `1:3` : `"integer"`, `"numeric"`, `"integer"`.

Les six types atomiques. `logical` · `integer` et `double` (tous deux de `mode numeric`) · `complex` · `character` · `raw`. « *Single numbers ... and strings ... **are still vectors, of length 1 ; there are no more basic types.*** »

Coercition. « *R caters for changes of mode **almost anywhere it could be considered sensible*** » · famille `as.quelquechose()` · **numérique** → **caractère** → **numérique** n'est pas exactement réversible (arrondis) · **la coercition efface tous les attributs.**

Longueur. Allonger : écrire **hors des bornes** (`e[3] <- 17`, les deux premiers deviennent `NA`), « *provided the mode of the **additional component(s) agrees*** » ; utilisé par `scan()`. **Raccourcir** : une assignation (`alpha <- alpha[2*1:5]`) ou `length(alpha) <- 3`, qui **allonge aussi**, par des `NA`.

Attributs. `attributes()` liste, `attr()` sélectionne, `attr<-` pose · « *rarely used ... **The concept, however, is very important*** » · **tout objet sauf `NULL` peut en porter** · **préférer les accesseurs dédiés** — ils **vérifient la cohérence** · retirer un attribut = lui assigner `NULL`.

Les cinq spéciaux. `names` — étiquette les éléments, sert à l'indexation (`quantile(x)["25%"]`) ; sur un tableau 1-D il **accède en fait à** `dimnames[[1]]`. `dim` — **implémente les tableaux**, contenu en **ordre colonne-majeur, longueur = produit des extensions**, une extension **peut être nulle** ; **un vecteur n'a pas de `dim`**, un tableau 1-D en a un **de longueur 1**. `dimnames` — **une liste de vecteurs de caractères**,

elle-même nommable (d'où les en-têtes). `class` — le vecteur des classes dont l'objet hérite, base des méthodes génériques, **modifiable sans contrôle** → prudence. `tsp` — **start, end, frequency**.

La phrase-clé. « *Matrices and arrays are simply vectors with the attribute `dim` and optionally `dimnames` attached to the vector.* »

Copie des attributs (Becker-Chambers-Wilks, pp. 144-6). Fonction **scalaire** → **préserve** (sauf peut-être `class`) · **binaire** → copie depuis **le plus long** (à égalité, **le premier**), sauf `names` / `dim` / `dimnames` que l'opérateur pose lui-même · **sous-ensemble** → **efface tout sauf** `names` / `dim` / `dimnames` · **sous-assignation** → **préserve**, même si la longueur change · **coercition** → **efface tout** · **tri** → efface tout sauf `names`, triés avec l'objet.

Classe. Tout objet a une classe ; pour un vecteur simple **c'est le mode** ; sinon `"matrix"`, `"array"`, `"factor"`, `"data.frame"` ... · elle commande **l'affichage**, `plot()`, `summary()` et les autres génériques · `unclass()` retire temporairement l'effet — « *one [use] is when you are **learning to come to terms with the idea of class*** » · **S4** vit dans le paquet `methods`.

Trois objets à connaître. `NULL` — **l'absence**, pas le vide ; **un seul** objet `NULL` ; **aucun type, aucun attribut possible** ; `is.null()` · **Environnements** — **frame + enclosure**, arbre enraciné dans `emptyenv()`, parent direct de `baseenv()` ; **non copiés** lors d'un passage à une fonction ou d'une assignation. **Promesses** — **valeur, expression, environnement** ; chaque argument formel y est lié ; **rien n'est évalué avant l'accès**, puis **le résultat est mémorisé** ; `substitute` lit la fente expression.

Objets composés. Facteur = attribut `levels` + classe `"factor"` (+ `contrasts` optionnel) ; ordonné → `c("ordered", "factor")` ; le codage entier est **un détail d'implémentation, non garanti**. **Data frame** = **une liste** de vecteurs, facteurs et/ou matrices **de même longueur**, avec `names` (variables) et `row.names` (cas) ; peut contenir une liste pour des **tableaux irréguliers**, « *not generally handled correctly* ».

Active Recall

« These are known as "**atomic**" structures since **their components are all of the same type, or mode** » — les cinq modes atomiques étant `numeric`, `complex`, `logical`, `character` et `raw` (§3.1).

*« **Lists** ... are known as "**recursive**" rather than atomic structures since **their components can themselves be lists in their own right**. ... The other recursive structures are those of mode `function` and `expression`. »*

Et la précision de la *R Language Definition* (§2.1.2) : « **Lists are vectors**, and the basic vector types are referred to as **atomic vectors** where it is necessary to exclude lists. »
Une liste **est** un vecteur — un vecteur générique.

```
x <- 1:3
typeof(x)      # "integer"
mode(x)        # "numeric"
storage.mode(x) # "integer"
```

La note 1 du §3.1 d'*An Introduction to R* le dit : « **numeric mode is actually an amalgam of two distinct modes, namely integer and double precision** ».

Et la *R Language Definition* (§2) explique le partage des rôles : `typeof` rend **le type R réel** (le `SEXPTYPE`) ; `mode` rend le type « *in the sense of Becker, Chambers & Wilks (1988)* », **plus compatible avec les autres implémentations de S** ; `storage.mode` sert « *when calling functions written in another language, such as C or FORTRAN* ».

Non : « **Vectors must have their values all of the same mode.** Thus any given vector must be **unambiguously** either logical, numeric, complex, character or raw. » (§3.1)

Le `NA` semble faire exception ; le cours répond : « *in fact **there are several types of NA*** ».

Quand on mélange, R **coerce** — « *R caters for changes of mode almost anywhere it could be considered sensible to do so* ». Le mode retenu est **le plus général capable de tout représenter : logical → integer → double → complex → character**. (Cette hiérarchie est une conséquence de la règle, pas une citation du cours.) D'où `c(1, "a")` qui vaut `c("1", "a")`.

Note 3 du §3.1 : « *In general, **coercion from numeric to character and back again will not be exactly reversible**, because of **roundoff errors in the character representation**.* »

L'aller-retour donné par le cours — `z <- 0:9 ; digits <- as.character(z) ; d <- as.integer(digits)` — fonctionne **parce que ce sont des entiers**, dont l'écriture décimale

est exacte. Un `double` passe par une représentation d'affichage tronquée, et le retour ne redonne pas exactement le même nombre.

Et il y a pire : « ***coercion drops all attributes*** » (§2.2.6). L'aller-retour perd aussi `dim`, `names`, `class` ...

L'exemple du §3.3 :

```
attr(z, "dim") <- c(10, 10)
```

« *allows R to **treat** `z` as if it were a 10-by-10 matrix* ». Rien n'a été copié ni converti : **un attribut a été posé**.

L'écriture préférable est `dim(z) <- c(10, 10)` : le §2.2 recommande les accesseurs dédiés, « *in addition to **hiding details of implementation** they may **perform additional operations*** », et R **contrôle la cohérence** — « *R ensures that **the length of the vector is the product of the lengths of the dimensions*** ».

Pour revenir en arrière : `dim(z) <- NULL` .

« *The content of the array is **stored in a vector in column-major order*** » (§2.2.2) : la matrice se remplit **colonne par colonne**. Pour `matrix(1:4, 2)` :

$$\begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Trois conséquences. (1) `m[2]` est l'élément **sous** `m[1]`, pas à sa droite. (2) `m[1]` a un sens et vaut `c(m)[1]` (fiche 302). (3) `length(m)` rend **le produit des extensions** — le nombre de **cases** —, jamais le nombre de lignes.

Les règles du §2.2.6 (Becker, Chambers & Wilks, pp. 144-6) :

Opération	Attributs
fonction scalaire	préservés (sauf peut-être <code>class</code>)
opération binaire	copiés depuis le plus long ; à égalité, le premier — sauf <code>names</code> / <code>dim</code> / <code>dimnames</code>
sous-ensemble	tout tombe sauf <code>names</code> , <code>dim</code> , <code>dimnames</code>
sous-assignation	préservés , même si la longueur change
coercition	tout tombe
tri	tout tombe sauf <code>names</code> , triés avec l'objet

Un attribut maison — une unité, une date de création — **ne survit donc pas à** `x[1:5]` , mais survit à `x[1:5] <- v` .

« *This attribute **can be accessed and manipulated virtually without restriction by users. There is no checking that an object actually contains the components that class methods expect.** Thus, **altering the** `class` **attribute should be done with caution, and when they are available specific creation and coercion functions should be preferred.*** » (§2.2.4)

Poser `class(x) <- "data.frame"` **ne convertit rien** : cela dit seulement à R de chercher les méthodes des data frames. Comme `x` n'a ni `names` ni `row.names` ni colonnes, la panne survient **au premier appel générique** — affichage, `summary()` , `plot()` — **loin de la ligne fautive**.

« ***To remove temporarily the effects of class, use the function** `unClass()` .* » Un data frame `winter` s'affiche « *in data frame form, which is rather like a matrix* » ; `unclass(winter)` l'affiche « ***as an ordinary list*** ».

« ***Only in rather special situations do you need to use this facility, but one is when you are learning to come to terms with the idea of class and generic functions.*** » (§3.4)

C'est l'outil qui montre qu'un **data frame est une liste**, qu'un **facteur est un vecteur d'entiers**, qu'une **matrice est un vecteur**. Avec `attributes()`, il dissipe toute la magie apparente des objets R.

« *There is a special object called `NULL`. It is used **whenever there is a need to indicate or specify that an object is absent. It should not be confused with a vector or list of zero length.** ... The `NULL` object **has no type and no modifiable properties. There is only one `NULL` object in R.** ... **You cannot set attributes on `NULL`.** » (§2.1.6)*

`NULL` = l'absence ; `numeric(0)` = un **vecteur numérique vide**, qui a un type et peut porter des attributs. Les deux ont `length()` nul.

C'est pourquoi `x$inexistent` rend `NULL` (fiche 302) : la composante **n'est pas là**, elle n'est pas vide. Le test est `is.null()`.

« *Unlike most other R objects, environments are not copied when passed to functions or used in assignments. Thus, **if you assign the same environment to several symbols and change one, the others will change too.** In particular, **assigning attributes to an environment can lead to surprises.** » (§2.1.10)*

C'est l'**exception** à la sémantique de copie que décrit le mécanisme ``*tmp*`` (fiche 302). Tout R se comporte **par valeur**, sauf les environnements, qui se comportent **par référence**.

Double conséquence : ils permettent délibérément l'effet de bord quand on en a besoin, et ils produisent des bugs incompréhensibles quand on ignore leur présence.

« ***Promise objects are part of R's lazy evaluation mechanism. They contain three slots : a value, an expression, and an environment. When a function is called ... each of the formal arguments is bound to a promise. The expression that was given for that formal argument and a pointer to the environment the function was called from are stored in the promise.** » (§2.1.8)*

« ***Until that argument is accessed there is no value associated with the promise. When the argument is accessed, the stored expression is evaluated in the stored environment ... The result is also saved by the promise.** »*

Deux conséquences pratiques : un argument jamais utilisé n'est **jamais évalué** — il peut donc être invalide sans conséquence ; et `substitute` peut retrouver **le texte** de ce qui a été passé, puisqu'il « *will extract the content of the expression slot* ».

« *There is **generally no way in R code to check whether an object is a promise*** ».

Facteur (§2.3.1) : *« A factor **has a `levels` attribute and class `"factor"`** . Optionally, it may also contain a `contrasts` attribute »*. Ordonné : « *it should be defined as such and have **a class vector `c("ordered", "factor")`*** ».

« *Factors are **currently implemented** using **an integer array** ... **Rather unfortunately users often make use of the implementation** ... **This, however, is an implementation issue and is not guaranteed to hold in all implementations of R.*** »

Data frame (§2.3.2) : « *A data frame is a list of vectors, factors, and/or matrices all **having the same length** (number of rows in the case of matrices). In addition, a data frame generally has a `names` attribute labeling **the variables** and a `row.names` attribute for labeling **the cases**.* »

Allonger : « *new components may be added to it **simply by giving it an index value outside its previous range*** » (§3.2). `e <- numeric()` puis `e[3] <- 17` donne un vecteur de **longueur 3**, « *the first two components of which are at this point **both NA*** ».

La condition : « ***provided the mode of the additional component(s) agrees with the mode of the object in the first place*** » — sinon, coercion de **tout** l'objet.

Raccourcir : « *to truncate the size of an object requires only an assignment* ». `alpha <- alpha[2*1:5]` garde les indices **pairs** (`2*1:5 = c(2,4,6,8,10)`), le `:` étant prioritaire ; `length(alpha) <- 3` garde les trois premiers, « *and vectors **can be extended (by missing values) in the same way*** ».

Ce mécanisme « *is used often, for example in the `scan()` function for input* ».

*« By the **mode** of an object we mean the basic type of its fundamental constituents. **This is a special case of a "property" of an object.** Another property of every object is **its `length`** . »* (§3.1)

« *Further properties of an object are usually provided by `attributes(object)` . Because of this, `mode` and `length` are also called "intrinsic attributes" of an object.* »

Autrement dit : `mode` et `length` sont des propriétés que **tout** objet possède **par construction**, tandis que les autres attributs sont **attachés**. C'est pourquoi `attributes(x)` ne les liste pas — il rend « *a list of all the **non-intrinsic** attributes* ».

« *`length(object)` **does not always contain intrinsic useful information**, e.g. when `object` is a function.* »

Flashcards

Question	Réponse
Comment R appelle-t-il ce sur quoi il opère ?	Des objets
Une structure atomique ?	Tous les composants du même mode
Les cinq modes atomiques ?	<code>numeric</code> · <code>complex</code> · <code>logical</code> · <code>character</code> · <code>raw</code>
Une structure récursive ?	Ses composants peuvent être n'importe quoi
Les trois modes rékursifs ?	<code>list</code> · <code>function</code> · <code>expression</code>
Une liste est-elle un vecteur ?	Oui — un « vecteur générique »
Comment appelle-t-on un vecteur non-liste ?	Un vecteur atomique
Un vecteur vide a-t-il un mode ?	Oui : <code>character(0)</code> , <code>numeric(0)</code>
Les deux attributs intrinsèques ?	<code>mode</code> et <code>length</code>
Pourquoi « intrinsèques » ?	Les autres viennent d' <code>attributes()</code>
<code>length()</code> est-il toujours informatif ?	Non — par exemple sur une fonction
Que rend <code>typeof()</code> ?	Le type réel (le <code>SEXPTYPE</code>)
Que rend <code>mode()</code> ?	Le type au sens de Becker-Chambers-Wilks
Que rend <code>storage.mode()</code> ?	Le mode de stockage — pour C et Fortran
<code>typeof(1:3)</code> ?	<code>"integer"</code>
<code>mode(1:3)</code> ?	<code>"numeric"</code>
Pourquoi cette différence ?	<code>numeric</code> amalgame <code>integer</code> et <code>double</code>
Les six types de vecteurs ?	<code>logical</code> <code>integer</code> <code>real</code> <code>complex</code> <code>character</code> <code>raw</code>
<code>4.2</code> est-il un scalaire ?	Non — un vecteur de longueur 1
La structure C sous-jacente ?	<code>SEXPREC</code> ; les types sont des <code>SEXPTYPE</code>
<code>typeof</code> d'une fonction ?	<code>"closure"</code>

Question	Réponse
<code>typeof</code> d'un argument non encore évalué ?	<code>"promise"</code>
Différence <code>builtin</code> / <code>special</code> ?	Le premier évalue ses arguments, le second non
La famille des fonctions de conversion ?	<code>as.quelquechose()</code>
Aller-retour numérique → caractère → numérique ?	Pas exactement réversible (arrondis)
Que fait la coercition aux attributs ?	Elle les efface tous
Comment allonger un vecteur ?	Écrire hors des bornes
Que valent les cases intermédiaires ?	<code>NA</code>
À quelle condition ?	Que le mode concorde
Quelle fonction utilise ce mécanisme ?	<code>scan()</code>
Comment tronquer ?	Une assignation , ou <code>length(x) <- n</code>
<code>length(x) <- n</code> peut-il allonger ?	Oui , par des valeurs manquantes
Lister les attributs ?	<code>attributes(x)</code>
En lire un seul ?	<code>attr(x, "nom")</code>
En retirer un ?	Lui assigner <code>NULL</code>
Qui peut porter des attributs ?	Tout sauf <code>NULL</code>
Comment sont-ils stockés ?	Comme une pairlist entièrement nommée
Pourquoi préférer <code>names<-</code> à <code>attr<-</code> ?	L'accesseur vérifie la cohérence
Les cinq attributs spéciaux ?	<code>names</code> · <code>dim</code> · <code>dimnames</code> · <code>class</code> · <code>tsp</code>
À quoi sert <code>names</code> ?	Étiqueter les éléments — et indexer
Exemple d'indexation par nom ?	<code>quantile(x)["25%"]</code>
<code>names</code> sur un tableau 1-D ?	Accède en fait à <code>dimnames[[1]]</code>
Qu'implémente <code>dim</code> ?	Les tableaux

Question	Réponse
Dans quel ordre le contenu est-il stocké ?	En colonne-majeur
Quelle contrainte R impose-t-il ?	<code>length</code> = produit des extensions
Une extension peut-elle être nulle ?	Oui
Vecteur contre tableau 1-D ?	Le second a un <code>dim</code> de longueur 1
Qu'est-ce que <code>dimnames</code> ?	Une liste de vecteurs de caractères
Que permet le nommage de cette liste ?	Les en-têtes d'extension à l'affichage
Qu'est-ce que <code>class</code> ?	Le vecteur des classes dont l'objet hérite
Que fonde-t-il ?	Les méthodes génériques
Est-il contrôlé ?	Non — « <i>there is no checking</i> »
Que contient <code>tsp</code> ?	start, end, frequency
La phrase sur les matrices ?	« <i>simply vectors</i> with the attribute <code>dim</code> ... attached »
Une fonction scalaire et les attributs ?	Elle les préserve (sauf peut-être <code>class</code>)
Une opération binaire ?	Copie depuis le plus long ; à égalité, le premier
Que pose l'opérateur lui-même ?	<code>names</code> , <code>dim</code> , <code>dimnames</code>
Un sous-ensemble ?	Efface tout sauf <code>names</code> / <code>dim</code> / <code>dimnames</code>
Une sous-assignation ?	Préserve , même si la longueur change
Un tri ?	Efface tout sauf <code>names</code> , triés avec l'objet
Tout objet a-t-il une classe ?	Oui
Classe d'un vecteur simple ?	Son mode
Quatre autres valeurs possibles ?	<code>"matrix"</code> <code>"array"</code> <code>"factor"</code> <code>"data.frame"</code>
Trois génériques sensibles à la classe ?	L' affichage , <code>plot()</code> , <code>summary()</code>
Comment voir dessous ?	<code>unclass()</code>

Question	Réponse
Quand le cours le recommande-t-il ?	Quand on apprend la notion de classe et de générique
Où vivent les classes S4 ?	Le paquet <code>methods</code>
Qu'est-ce que <code>NULL</code> ?	L'absence d'objet
Combien y en a-t-il ?	Un seul , partagé
A-t-il un type ?	Non , ni de propriété modifiable
Peut-il porter un attribut ?	Non
Comment le tester ?	<code>is.null()</code>
De quoi ne faut-il pas le confondre ?	D'un vecteur ou d'une liste de longueur nulle
De quoi est fait un environnement ?	D'un frame et d'une enclosure
La racine de l'arbre ?	<code>emptyenv()</code>
Son enfant direct ?	L'environnement du paquet base (<code>baseenv()</code>)
Les environnements sont-ils copiés ?	Non — les seuls objets dans ce cas
Les trois fentes d'une promesse ?	valeur, expression, environnement
Quand la valeur est-elle calculée ?	À l'accès , puis mémorisée
Quelle fonction lit la fente expression ?	<code>substitute</code>
Peut-on tester si un objet est une promesse ?	Généralement non
Les deux attributs d'un facteur ?	<code>levels</code> et <code>class = "factor"</code>
Un facteur ordonné ?	Classe <code>c("ordered", "factor")</code>
Peut-on compter sur son codage entier ?	Non — « <i>an implementation issue</i> »
Qu'est-ce qu'un data frame, au fond ?	Une liste de vecteurs de même longueur
Ses deux attributs de nommage ?	<code>names</code> (variables) et <code>row.names</code> (cas)
Quelle structure R imite-t-il ?	Le jeu de données SAS ou SPSS — « cas par variables »